Analytics Engineering: Session 2

Setting Up dbt & Building First Models

Agenda

- Introduction to DuckDB
- Setting up the environment and database
- Git workflow: forking & cloning
- The VS Code IDE & dbt Power User
- The dbt project structure (`dbt_project.yml`)
- Building your first model
- The `ref()` macro & modularity
- Key dbt commands
- Compiled vs run SQL (`target/`)
- Naming conventions

Introduction to DuckDB

Why DuckDB?

- **"SQLite for analytics"**: an in-process SQL OLAP database.
- **Serverless**: no server to install, configure, or maintain.
- **Fast**: columnar storage, vectorized execution — optimized for analytical queries.
- **Integrated**: reads Parquet/CSV directly, works natively with Python and dbt.

In this course, **DuckDB acts as our Data Warehouse**, running locally on your machine. The file `my_database.duckdb` *is* our warehouse.

Setting up the Environment

We use `uv` as the Python package manager — fast and reproducible.

```
uv sync                          # install deps, create .venv/
source .venv/bin/activate        # macOS / Linux
.\.venv\Scripts\Activate.ps1     # Windows PowerShell
dbt deps                         # install dbt packages
```

Tip: skip activation and prefix with `uv run`, e.g. `uv run dbt debug`.

Setting up the Database

We load the raw Parquet files into DuckDB with a helper script:

```
python create_db.py
```

This creates `my_database.duckdb` with 14 raw tables: `customers`, `orders`, `order_items`, `products`, `shipping`, `payments`, `reviews`, and more.

Verify the dbt connection:

```
dbt debug
```

If everything is green, you are ready to build.

How dbt Finds Your Profile

dbt looks for `profiles.yml` — **first match wins**:

1. `--profiles-dir /path` (CLI flag)
2. `DBT_PROFILES_DIR` env var
3. **The project directory** (next to `dbt_project.yml`) ← *this repo*
4. `~/.dbt/profiles.yml` (global default)

```yaml
# profiles.yml — ships at the repo root
default:
  target: dev
  outputs:
    dev: {type: duckdb, path: my_database.duckdb, schema: main}
```

`dbt debug` prints the exact path it loaded — check that first if anything looks off. We skip `dbt init` because the repo already ships everything you need.

Git Workflow: Forking & Cloning

All your work lives inside **your own fork** — nothing is pushed to the instructor's repo.

1. **Fork** `dgarhdez/dbt-ie` on GitHub → `YOUR_USERNAME/dbt-ie`.
2. **Clone**: `git clone https://github.com/YOUR_USERNAME/dbt-ie.git`
3. **Branch**: `git checkout -b feature/my-first-model`
4. **Commit, push, open a PR** (feature branch → your own `main`) and merge it yourself.

Same PR workflow as industry — just scoped to your fork.

The VS Code IDE

We use **Visual Studio Code** as our Integrated Development Environment.

Recommended extensions:

- **Python** — run scripts and manage environments.
- **dbt Power User** (by Innoverio) — dbt-specific features.
- **SQLTools** + **SQLTools DuckDB driver** — query the warehouse directly.
- **Rainbow CSV** — readable CSV previews.

The dbt Power User Extension

Bridges the gap between your code and dbt execution.

Install: Extensions view (`Cmd+Shift+X`) → search "dbt Power User" → Install.

Features you will use daily:

- **Go to Definition** — `Cmd+Click` on a `ref()` to jump to the referenced model.
- **Query Preview** — run the current model's SQL against DuckDB without a full `dbt run`.
- **Compiled SQL panel** — see the compiled output live.
- **Lineage view** — visualize upstream/downstream dependencies.

The dbt Project Structure

Every dbt project is defined by `dbt_project.yml` at the repo root.

```yaml
name: 'dbt_ie'
profile: 'default'          # profile from profiles.yml

model-paths: ["models"]
seed-paths:  ["seeds"]

models:
  dbt_ie:
    staging:
      materialized: view    # default for models/staging/
```

The `models:` section configures defaults **by folder**, cascading down.

Project Directory Layout

```
dbt-ie/
├── dbt_project.yml          # project config
├── packages.yml             # dbt packages to install
├── models/                  # your transformations (SQL + Python)
│   ├── staging/
│   ├── intermediate/
│   └── marts/
├── seeds/                   # static CSVs loaded as tables
├── macros/                  # reusable Jinja snippets
├── snapshots/               # SCD Type 2 tracking
├── tests/                   # singular (custom) tests
├── analyses/                # ad-hoc SQL, never executed by `dbt run`
├── target/                  # compiled + run artifacts (gitignored)
└── logs/                    # dbt run logs (gitignored)
```

Building Your First Model

A **dbt model** is a `SELECT` saved as a `.sql` file under `models/`.

```
-- models/staging/stg_customers.sql
select customer_id, first_name, last_name, email, country
from main.customers
```

Build it: `dbt run -s stg_customers`

We read directly from `main.customers` for now — next session we wrap raw tables with `source()`.

The `ref()` Macro & Modularity

`ref()` is how models reference each other — the core of dbt's **DAG**.

```
-- models/intermediate/int_customer_orders.sql
select c.customer_id, c.first_name, count(o.order_id) as n_orders
from {{ ref('stg_customers') }} as c
left join {{ ref('stg_orders') }} as o on c.customer_id = o.customer_id
group by 1, 2
```

- Resolves to the correct schema/table **at compile time**.
- Builds a **Directed Acyclic Graph** and the correct execution order.

Why `ref()` Matters

Without `ref()`:

```
-- brittle: hardcoded schema, no dependency tracking
from main.stg_customers
```

With `ref()`:

```
from {{ ref('stg_customers') }}
```

- **Portable**: same code works across dev, staging, and prod schemas.
- **DAG-aware**: dbt knows to build `stg_customers` *before* any model that refs it.
- **Safe refactoring**: rename a model → dbt flags every downstream consumer.

Key dbt Commands

Command	Purpose
`dbt debug`	Verify connection and project config
`dbt deps`	Install packages from `packages.yml`
`dbt compile`	Compile Jinja → SQL, no execution
`dbt run`	Execute all models
`dbt run -s +mart_orders`	Build `mart_orders` and everything upstream
`dbt build`	`run` + `test` + `seed` + `snapshot` in DAG order

Selectors (`-s`) support `+`, `@`, `tag:`, `path:`, and more.

Compiled vs Run SQL

After `dbt compile` or `dbt run`, dbt writes two outputs under `target/`:

- `target/compiled/` — SQL **after Jinja rendering**, before execution. *What dbt would send to the warehouse.*
- `target/run/` — SQL **wrapped in materialization DDL** (e.g. `create table as ...`). *What dbt actually executed.*

```
target/compiled/dbt_ie/models/staging/stg_customers.sql   # pure SELECT
target/run/dbt_ie/models/staging/stg_customers.sql        # CREATE VIEW ... AS SELECT
```

Always check compiled SQL when debugging — it is the ground truth.

Naming Conventions

Consistent naming helps teams collaborate and keeps the DAG readable.

Layer	Prefix	Example
Staging	`stg_`	`stg_customers`, `stg_orders`
Intermediate	`int_`	`int_order_items_summary`
Dimensions	`dim_`	`dim_customers`, `dim_products`
Marts / Facts	`mart_`	`mart_orders`, `mart_revenue_by_segment`

We will apply these consistently for the rest of the course — they map directly to the three-layer architecture introduced in Session 1.

What have we learned in this session

- Set up the environment with `uv sync` and loaded data into DuckDB
- Forked and cloned the course repo; understood the PR workflow
- Configured VS Code with the dbt Power User extension
- Navigated the dbt project structure and `dbt_project.yml`
- Built our first model and used `ref()` to create a dependency
- Learned the core dbt commands: `debug`, `deps`, `compile`, `run`, `build`
- Inspected compiled vs run SQL in `target/`

Next session: Sources, Seeds, Dependencies & First Tests.